

Kolokwium 1 - Java

Imię:

Nazwisko:

Na napisanie testu jest 85 minut. Każde poprawnie rozwiązane zadanie to jeden punkt. Każde niepoprawnie rozwiązane zadanie to zero punktów (dotyczy to również zadań z wielokrotnym wyborem)

1. Dla podanego kodu:

```
public class Zadanie<T> {  
    public <S extends T> void test(Collection<? extends T> a,  
        Collection<S> b, Collection<T> c){  
        XXX;  
    }  
}
```

zakreśl polecenia które nie spowodują błędu kompilacji w linii XXX:

a.addAll(a);	a.addAll(b);	a.addAll(c);
b.addAll(a);	b.addAll(b);	b.addAll(c);
c.addAll(a);	c.addAll(b);	c.addAll(c);

Zanalizuj poniższe kody (zakładamy że wszystkie importy są w porządku) i:

- jeśli nastąpi błąd kompilacji klasy Zadanie, to wskaż w którym miejscu,
- jeśli klasa Zadanie się skompiluje, ale po uruchomieniu wyrzucony zostanie wyjątek to wskaż w której linii,
- jeśli kod się wykona bez wyjątku to podaj wynik uruchomienia klasy Zadanie.

```
2. public class Zadanie {  
    static class A{ public A(String s) {System.out.print(s); } }  
    static class B{ static A a = new A("1"); }  
    static A a = new A("2");  
    public Zadanie() { a = new A("3"); }  
    public static void main(String[] args){ }  
}
```

```
3. public enum Zadanie {  
    ONE { void t() { System.out.println("1"); } },  
    TWO { void t() { System.out.println("2"); } };  
    public static void main(String[] args){  
        System.out.println(ONE.getClass() == TWO.getClass());  
        for(Zadanie temp : values()) temp.t();  
    }  
}
```

```

4. public class Zadanie {
    public static void f() { throw new RuntimeException(); }
    public static void main(String[] args){
        try{
            try { f(); } finally {}
        } catch(Exception e) { System.out.print("1");}
        try{
            try { f(); } finally {return;}
        } catch(Exception e) { System.out.print("2");}
    }
}

5. public class Zadanie {
    public static int t(Object[] o) { return o.length; }
    public static int t(Collection<Object> o) { return o.size(); }
    public static void main(String[] args) {
        t(new Integer[0]);
        t(new Object[0]);
        t(new ArrayList<Integer>());
        t(new ArrayList<Object>());
    }
}

6. interface ZadaniePrim<T> { Object t(T t); }
    public class Zadanie implements ZadaniePrim<Integer> {
        public Object t(Integer i) { return i + 1; }
        public Object t(Object o) { return o; }
        public static void main(String[] args){
            System.out.println(new Zadanie().t(0));
        }
    }

7. public class Zadanie {
    public static void main(String[] args) {
        LinkedList<Object[]> temp1 = new LinkedList<Object[]>();
        temp1.add(new Object[]{1,2,3});
        temp1.add(new Integer[]{1,2,3});
        temp1.add(new String[2]);
        for(Object[] a : temp1) System.out.print(Arrays.toString(a));
        LinkedList<List<Object>> temp2 = new LinkedList<List<Object>>();
        temp2.add(Arrays.asList(new Object[]{1,2,3}));
        temp2.add(Arrays.asList(new Integer[]{1,2,3}));
        temp2.add(Arrays.asList(new String[2]));
        for(List<Object> a : temp2) System.out.print(a);
    }
}

```

8.

```
public class Zadanie {  
    static class Comp implements Comparator<String>{  
        public int compare(String a, String b) { return b.compareTo(a); }  
    }  
    public static void main(String[] args){  
        TreeSet<String> temp = new TreeSet<String>(new Comp());  
        temp.addAll(Arrays.asList("Ala", "ma", "kota"));  
        System.out.println(temp);  
    }  
}
```
9. Czy podany poniżej komparator jest dobrze zaprojektowany?

```
public class Zadanie implements Comparator<Object>{  
    public int compare(Object o1, Object o2) {  
        return o1.hashCode() < o2.hashCode() ? 1 :  
            o1.hashCode() == o2.hashCode() ? 0 : -1;  
    }  
}
```
- TAK
 - NIE (podaj uzasadnienie)
10. Podaj przykłady dobrze ilustrujące różnice pomiędzy:
- kompozycją a dziedziczeniem,
 - metodą statyczną a metodą instancji,
 - finalną metodą a finalnym polem.
11. Kiedy nadpisujemy metodę equals(...) należy pamiętać że:
- -
 -
 -
12. W którym miejscu poniższej metody obiekt zainicjowany w linii 1 może zostać (najwcześniej !) odświeżony:
- ```
void method() {
 String a = new String("aaa"); // 1
 a = a+1; //2
 a = null; //3
} //4
```
- przed wykonaniem 2
  - przed wykonaniem 3
  - przed wykonaniem 4
  - nigdy

13. Dla poniższego kodu

```
public class Zadanie {
 public class I {public String toString() { return "2"; } }
 public class J extends I {public String toString() { return XXX; } }
 public J gimme() { return new J(); }
 public String toString() { return "4"; }
 public static void main(String[] args){
 System.out.println(new Zadanie().gimme());
 }
}
```

Jaki tekst należy wstawić w miejsce oznaczone XXX aby:

- o wykonać metodę toString dla obiektu rodzica,
- o wykonać metodę toString dla obiektu zawierającego obiekt J.

14. Jeśli klasa A ma metodę private void t() {} to dziedzicząca po A klasa B może mieć metodę void t() {} poprzedzoną (wypisz wszystkie dopuszczalne modyfikatory dostępu):

Jeśli klasa A ma metodę void t() {} (bez modyfikatora dostępu) to dziedzicząca po A klasa B może mieć metodę void t() {} poprzedzoną (wypisz wszystkie dopuszczalne modyfikatory dostępu):

15. Dany jest kod:

```
public class Zadanie<T extends Throwable> {
 Zadanie() throws T { }
 void t() throws T { };
}
class ZadaniePrim extends Zadanie<Exception> {
 XXX;
 YYY;
}
```

Czy w miejscu XXX może się pojawić (tak/nie):

- o ZadaniePrim() throws Throwable { }
- o ZadaniePrim() throws RuntimeException { }

Czy w miejscu YYY może się pojawić (tak/nie):

- o public void t() throws RuntimeException { }
- o public void t() throws Throwable { }

16. Klasa A<T> jest zdefiniowana następująco:

```
class A<T> { T id(T t) { return t; } }
```

zaznacz linie które skompilują i wykonają się poprawnie:

- ☐ A<Integer> a = new A<Integer>(); Integer i = a.id(7);
- ☐ A<? extends Integer> b = new A<Integer>(); Integer j = b.id(7);
- ☐ A<? super Integer> c = new A<Integer>(); Object k = c.id(7);
- ☐ A<?> d = new A<Integer>(); Object l = d.id(null);

17. Refleksja w Java umożliwia (zakreśl poprawne kontynuacje zdania):

- o operowanie na metodach klas w trakcie działania kodu,
- o identyfikację klas podanych jako parametry typów (dla typów uogólnionych),
- o identyfikację klasy obiektu w trakcie wykonania kodu,
- o identyfikację wszystkich interfejsów które implementuje ta klasa.

18. Jeśli klasa A posiada metodę

```
public String t(String s) { return "A"; }
```

to klasa B dziedzicząca po A może posiadać:

- ☐ `public String t(Object o) { return "B"; }`
- ☐ `String t(String s) { return "B"; }`
- ☐ `public Object t(String s) { return "B"; }`
- ☐ `Integer t(String s) { return 2; }`
- ☐ `public Integer t(Integer i) { return 2; }`

19. Jaki jest wynik skompilowania i uruchomienia poniższego kodu:

```
public class Singleton {
 private static Singleton TheOne = new Singleton();
 private Singleton(){
 System.out.println("I am the One.");
 }
 public static void main(String[] args) {
 Singleton x = new Singleton();
 }
}
```

- ☐ Błąd kompilacji. Próba uruchomienia konstruktora prywatnego przy tworzeniu obiektu x.
- ☐ Poprawna kompilacja. Wyjątek w trakcie działania przy próbie utworzenia obiektu x.
- ☐ Poprawna kompilacja. Na ekranie pojawi się napis "I am the One.", a x będzie równy null.
- ☐ Poprawna kompilacja. Zostaną utworzone dwa obiekty Singleton (na ekranie pojawią się dwa napisy "I am the One."

20. Jeśli składową klasy nie poprzedzimy operatorem dostępu to domyślnie dostęp do niej jest:

- ☐ `private`
- ☐ `protected`
- ☐ `public`
- ☐ żaden z powyższych

21. Przy których metodach poniższego kodu pojawi się błąd kompilacji?

```
interface face {
 void one();
 public void two();
 protected void three();
 private void four();
}
```

- ☐ one, two, three, four
- ☐ one
- ☐ four
- ☐ three, four
- ☐ nie pojawi się błąd kompilacji

22. Podaj wynik skompilowania i uruchomienia poniższego kodu.

```
public class Wyjatki {
 public static void main(String[] args) {
 try {
 int a = 2012 / 0;
 } catch (Exception e) {
 System.out.println("Exception");
 } catch (ArithmeticException e) {
 System.out.println("ArithmeticException");
 } finally {
 System.out.println("finally");
 }
 }
}
```

- Błąd kompilacji
- Kompilacja poprawna. Błąd uruchomienia.
- Kompilacja poprawna. Wypisuje Exception oraz finally.
- Kompilacja poprawna. Wypisuje ArithmeticException oraz finally.
- Żaden z powyższych.

23. Podana jest następująca klasa

```
public class Outside {
 private class Inside{Inside(){}
 Outside(){}
}
```

W celu umożliwienia utworzenia obiektu klasy wewnętrznej Inside można posłużyć się instrukcją:

- new Outside.Inside();
- new Outside().new Inside();
- Outside.new Inside();
- nigdy nie uda się utworzyć obiektów klasy Inside bo jest ona prywatna

24. Co zostanie wypisane po wykonaniu poniższego programu?

```
class A{}
class B extends A{}
class C extends B{}
public class Polimorfizm {
 static void mimi(A x){System.out.print("A");}
 static void mimi(B x){System.out.print("B");}
 static void mimi(C x){System.out.print("C");}
 public static void main(String[] args) {
 B x = new C();
 mimi(x);
 mimi(null);
 }
}
```

○ AA

○ BA

○ CA

○ AC

○ BC

○ CC

25. Załóżmy, że klasa A dziedziczy po klasie B, a klasa B dziedziczy po klasie C. Wszystkie klasy implementują polimorficznie metodę `hello()`. Zaznacz instrukcje które uruchomią metodę `hello()` z klasy C jeśli wywołane zostaną w obiekcie klasy A:
- `hello();`
  - `super.hello();`
  - `super.super.hello();`
  - `C.hello();`
  - żadne z powyższych
26. Które deklaracje zakończą się błędem kompilacji:
- ```
class Z {  
    abstract class A {} // 1  
    final class B {}    // 2  
    private class C {}  // 3  
    protected class D {} // 4  
    public class E {}    // 5  
}
```
- 1
 - 2
 - 3
 - 4
 - 5
 - żadna
27. Które z poniższych linii są legalne?
- `int T[4] = {1,2,3,4};`
 - `int[] T[] = new int[4][4];`
 - `int T[3];`
 - `int[] T = new int[4] {1,2,3,4};`
 - `int[] T = new int[] {1,2,3,4};;`
28. Zaznacz wszystkie poprawne deklaracje metody `main`.
- `public static void main(String[] args){}`
 - `public void main(String[] args){}`
 - `public static void main(){}`
 - `public static void main(String... args){}`
 - `static void main(String[] args){}`

29. Jaki jest wynik działania poniższego programu?

```
public class Anon {  
    interface Inside {void print();}  
    public static void main(String[] args) {  
        new Anon().doSomething(5);  
    }  
    public void doSomething(int i) {  
        Inside t = new Inside() {  
            public void print() {  
                for (int j=0;j<=i;j++){System.out.print(" " + j);}  
            }  
        };  
        t.print();  
    }  
}
```

- a. Wyświetla 0 1 2 3 4 5
- b. Wyświetla 0 1 2 3 4
- c. Błąd kompilacji.
- d. Kompilacja poprawna. Błąd uruchomienia.

30. Jak jest wynik działania poniższego programu? Można założyć, że instrukcja System.gc() spowoduje odśmiecenie wszystkich nieużywanych obiektów wraz z uruchomieniem metod finalize().

```
public class MyExceptions {  
    public MyExceptions(){  
        throw new RuntimeException("constructor");    }  
    public void finalize(){  
        System.out.println("finalize");  
        throw new RuntimeException("finalize");  
    }  
    public static void main(String[] args){  
        try {new MyExceptions();}  
        catch (RuntimeException e) {System.out.println(e);}  
        System.gc();  
    }  
}
```

- a. Błąd kompilacji.
- b. W trakcie działania pojawi się wyjątek rzucony przez metodę finalize.
- c. Zostanie wyświetlone:
java.lang.RuntimeException: constructor
finalize
- d. Zostanie wyświetlone:
java.lang.RuntimeException: finalize